

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



LSI Logic Design

COURSE ID: CO3098 - HK252 CLASS: CC04

LSI Logic Design Lab

Lab 3 Cubic Solver

Lecturer: Tôn Huỳnh Long
Students: Chu Lê Hoàng - 2352346
Nguyễn Thế Khang - 2352490
Lương Đức Hiếu - 2352324
Nguyễn Minh Hy - 2352457
Nguyễn Bùi Tuấn Anh - 2352038

HỒ CHÍ MINH CITY, May 2026



Design Specification Summary

Field	Value
Project Name	Cubic Solver Module
Version	1.0
Date	2026/05/29
Author(s)	CC04 Group 1
Language	Verilog HDL

Table 1: Design Specification Overview

Contents

Design Specification Summary	1
1 Interface	3
2 Functional Implementation	5
2.1 System Overview	5
2.2 Mathematical Method	5
2.2.1 Normalization and Depression	5
2.2.2 Discriminant and Branch Selection	5
2.2.3 Cardano Branch ($D > 0$)	6
2.2.4 Trigonometric Branch ($D < 0$)	6
2.2.5 Repeated-Root Branch ($D = 0$)	6
2.2.6 Recovery of the Original Roots	7
2.3 Operation	7
2.3.1 Idle and Launch	7
2.3.2 Normalization and Coefficient Build-up	7
2.3.3 Discriminant and Dispatch	7
2.3.4 Branch Execution	7
2.3.5 Unshift and Completion	8
3 Internal Implementation	9
3.1 Overall	9
3.2 State Machine	10
4 Floating-Point Library	12
4.1 Adder / Subtractor (<code>fp32_add</code>)	12
4.2 Multiplier (<code>fp32_mul</code>)	12
4.3 Divider (<code>fp32_div</code>)	13
4.4 Square Root (<code>fp32_sqrt</code>)	13
4.5 Cube Root (<code>fp32_cbrt</code>)	13
4.6 Cosine (<code>fp32_cos</code>)	14
4.7 Arccosine (<code>fp32_acos</code>)	14
4.8 Exponential (<code>fp32_exp2</code>)	14
4.9 Logarithm (<code>fp32_log2</code>)	14

1 Interface

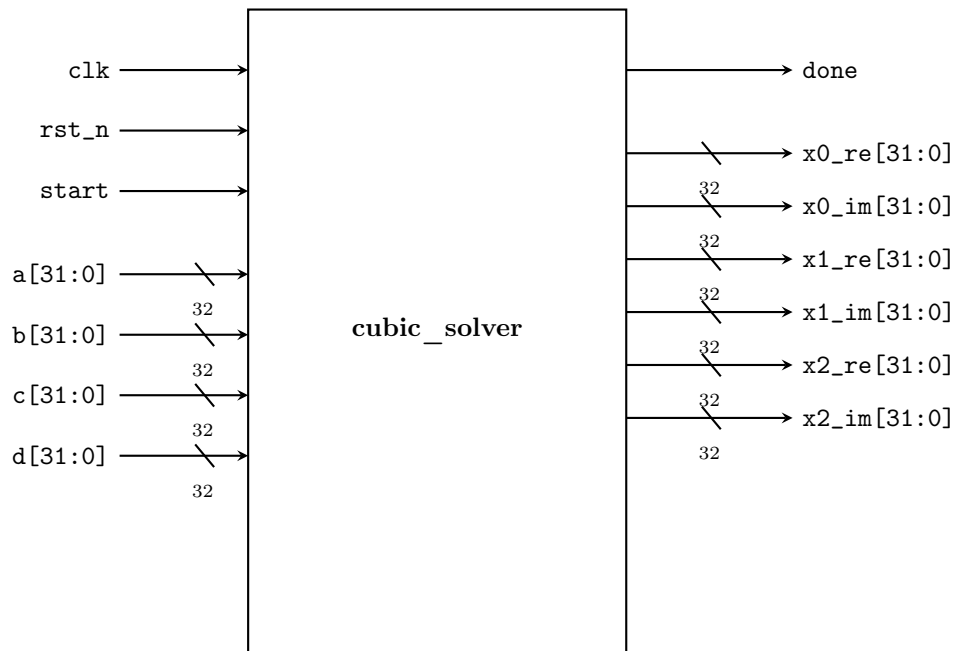


Figure 1: Block diagram of the `cubic_solver` module

The `cubic_solver` computes the three roots of the cubic equation $ax^3 + bx^2 + cx + d = 0$, where the coefficients and the roots are all represented in IEEE-754 single precision (FP32) format. The module is a synchronous, multi-cycle design: every sequential element is clocked on the rising edge of `clk`, and `rst_n` is an active low reset that returns the internal FSM to its idle state and clears every result in the register. A new solve is started by pulsing `start` HIGH for one clock cycle while the module is idle; the coefficients present on `a`, `b`, `c`, and `d` on that same edge and held for the remainder of the computation, so the caller doesn't need to keep them stable afterwards.

Because the algorithm shares a single instance of each floating point unit across many sequential steps, a solve occupies a fixed number of clock cycles. The `done` flag marks the end of that sequence: it is asserted for exactly one cycle, and on that cycle all six root components are simultaneously valid. The three roots are produced as Cartesian pairs, each consisting of a real part `xk_re` and an imaginary part `xk_im`; a purely real root carries a zero imaginary component, while complex roots appear as a pair sharing the same real part with opposite sign imaginary parts.

The degenerate case in which the leading coefficient `a` is zero (or is encoded as an infinity or NaN) does not describe a cubic, so the module does not start a solving iteration. Instead it drives all six output components to a NaN, signalling to the caller that no valid root set exists. This is the only input condition handled specially at the interface level; all other numerical edge cases are absorbed inside the datapath.

Signal	Width	Dir	Description
clk	1	In	System clock. All FSM state transitions and every result-capture register update occur on its rising edge.
rst_n	1	In	Active-low asynchronous reset. While LOW the FSM is forced to its idle state and all intermediate and output registers are cleared to zero.
start	1	In	Launch strobe. A single-cycle HIGH pulse, recognised only while the module is idle, latches the four coefficients and begins a solve. Pulses asserted mid-solve are ignored.
a[31:0]	32	In	FP32 cubic coefficient (leading term x^3). If a is zero or subnormal, Inf, or NaN the equation is not a cubic and all outputs are forced to NaN.
b[31:0]	32	In	FP32 coefficient of the x^2 term.
c[31:0]	32	In	FP32 coefficient of the x^1 term.
d[31:0]	32	In	FP32 constant term.
done	1	Out	Completion flag. Asserted HIGH for exactly one clock cycle when the six root components are valid; LOW at all other times.
x0_re[31:0]	32	Out	Real part of the first root. For the one real two complex case this is the only real root.
x0_im[31:0]	32	Out	Imaginary part of the first root. Zero whenever the root is real.
x1_re[31:0]	32	Out	Real part of the second root.
x1_im[31:0]	32	Out	Imaginary part of the second root.
x2_re[31:0]	32	Out	Real part of the third root.
x2_im[31:0]	32	Out	Imaginary part of the third root. When roots 1 and 2 are a complex pair, x2_im is the negation of x1_im .

Table 2: Port description of the `cubic_solver` module

2 Functional Implementation

2.1 System Overview

The `cubic_solver` module computes all three roots of a general cubic equation

$$ax^3 + bx^2 + cx + d = 0$$

with every coefficient and every root expressed in IEEE-754 single precision (FP32) format. The design follows the classical case analysis of the cubic: it reduces the general equation to a simpler form, inspects the sign of the discriminant, and then dispatches to one of three solution methods, each of which is exact for the family of cubics that falls into its branch. This structure is what allows a single hardware block to return in the correct format both the all real and the mixed real complex root sets that a cubic can produce.

The arithmetic is built entirely from a small library of pipelined FP32 functions (or submodules): an adder/subtractor, a multiplier, a divider, a square root, a cube root, a cosine, and an arccosine. Each is a fixed latency pipeline, 4 cycles for the adder and multiplier, 28 cycle for the square root and cube root, 40 for the cosine, 49 for the divider, and 68 for the arccosine. The solver itself contains no floating point logic of its own; it is a sequencer that presents operands to whichever unit a step needs, waits for that unit's latency to elapse, and captures the result before advancing. The divider, cosine, and arccosine are approximations: the divider uses Newton-Raphson and the cosine and arccosine use truncated polynomials so each introduces a small error; the adder, multiplier, square root, and cube root are effectively correctly rounded (nearest even).

The module is driven by a single clock `clk` with an active low reset `rst_n`. A solve is requested by pulsing `start` for one cycle, which latches the coefficients; the result becomes available some fixed number of cycles later, flagged by a cycle `done` pulse. The three roots are returned as real/imaginary FP32 pairs on `x0_re/x0_im` through `x2_re/x2_im`.

2.2 Mathematical Method

2.2.1 Normalization and Depression

The first step divides through by the leading coefficient to obtain the monic coefficients

$$b_a = \frac{b}{a}, \quad c_a = \frac{c}{a}, \quad d_a = \frac{d}{a}.$$

This division is the reason the case $a = 0$ must be rejected: it would correspond to a division by zero and, more fundamentally, to an equation that is not a cubic. The substitution

$$x = t - \frac{b_a}{3}$$

removes the quadratic term and yields the *depressed* cubic

$$t^3 + pt + q = 0,$$

with

$$p = c_a - \frac{b_a^2}{3}, \quad q = \frac{2b_a^3}{27} - \frac{b_a c_a}{3} + d_a.$$

The quantity $b_a/3$ is retained as a shift constant, once the depressed roots t_k are known, the roots of the original equation are recovered by the inverse substitution $x_k = t_k - b_a/3$.

2.2.2 Discriminant and Branch Selection

The nature of the roots is determined by the discriminant of the depressed cubic,

$$D = \frac{q^2}{4} + \frac{p^3}{27}.$$

This is the discriminant of the depressed cubic in Cardano form; its sign is opposite to the polynomial discriminant Δ used in the assignment brief, so $D > 0$ here corresponds to the brief's $\Delta < 0$ (one real root and a complex pair), and $D < 0$ to $\Delta > 0$ (three distinct real roots).

Its sign selects one of three solution methods:

- $D > 0$: one real root and a complex conjugate pair. Solved by *Cardano's* method.
- $D < 0$: three distinct real roots. Solved by the *trigonometric* method, which avoids the complex intermediate values that Cardano's formula would otherwise produce in this regime.
- $D = 0$: a repeated root. The cubic has either a double root plus a simple root, or a triple root. Solved by a direct closed form expression.

The branch decision is made by examining the sign and zero of the FP32 discriminant directly through its bit fields.

2.2.3 Cardano Branch ($D > 0$)

With $D > 0$ the real root is obtained from the two cube root terms

$$u = \sqrt[3]{-\frac{q}{2} + \sqrt{D}}, \quad v = \sqrt[3]{-\frac{q}{2} - \sqrt{D}},$$

giving the real root $t_0 = u + v$. The remaining two roots form a complex conjugate pair whose real part is $-\frac{1}{2}(u + v)$ and whose imaginary part is $\pm \frac{\sqrt{3}}{2}(u - v)$:

$$t_{1,2} = -\frac{u + v}{2} \pm j \frac{\sqrt{3}}{2}(u - v).$$

The real root is emitted with a zero imaginary part, and the conjugate pair is emitted as two outputs sharing a real part and carrying opposite sign imaginary parts.

2.2.4 Trigonometric Branch ($D < 0$)

When $D < 0$ all three roots are real, and Cardano's cube roots would act on complex arguments. The trigonometric identity for the cubic sidesteps this. The solver computes a magnitude

$$m = 2\sqrt{-\frac{p}{3}},$$

and an angle

$$\theta = \arccos\left(\frac{3q}{pm}\right),$$

after which the three real roots are

$$t_k = m \cos\left(\frac{\theta}{3} - k \frac{2\pi}{3}\right), \quad k = 0, 1, 2.$$

The first root uses the bare angle $\theta/3$; the second subtracts $2\pi/3$ and the third adds $2\pi/3$. All three carry a zero imaginary part. The argument to $\arccos$ stays within $[-1, 1]$ for any genuine $D < 0$ case, and the `fp32_acos` unit clamps marginal out-of-range values that round-off may produce near the $D = 0$ boundary.

2.2.5 Repeated-Root Branch ($D = 0$)

When $D = 0$ the roots collapse. Writing

$$r = \sqrt[3]{-\frac{q}{2}},$$

the depressed cubic has a simple root $t = 2r$ and a double root $t = -r$:

$$t_0 = 2r, \quad t_1 = t_2 = -r.$$

The double root is emitted twice so that the output always presents three roots regardless of branch.

2.2.6 Recovery of the Original Roots

In every branch the computed t_k are roots of the depressed cubic. The final stage applies the inverse shift,

$$x_k = t_k - \frac{b_a}{3},$$

to the real part of each root; the imaginary parts are unaffected by the real shift and pass through unchanged. The shifted values are what appear on the output ports.

2.3 Operation

The module operates as a multi-cycle engine wrapped around a single shared copy of each fp unit. A finite state machine walks through the computation one elementary operation at a time: in each state it drives the operands for whichever unit that step needs and then stalls, holding those operands steady while a latency counter counts the cycles until that unit's pipeline has produced a valid result; on that cycle it captures the result into a register and advances. Only one operation is in flight at a time and the units are shared, not streamed so the cost of each step is the full latency of the unit in use, and a complete solve takes several hundred cycles. This trades latency for area, since a fully parallel datapath would replicate the expensive square root, cube root, and trigonometric units several times over and cause the Genus synthesis tool to take several hours (our version 1.0 of the design took around 6 hours totally to fully synthesize but with this optimization we only took around 30 minutes thanks to Tuan Anh's pipeline analysis/implementation)

2.3.1 Idle and Launch

At reset, and whenever no solve is in progress, the FSM rests in its idle state with **done** held LOW. It remains there until **start** is sampled HIGH on a rising clock edge. On that edge the four coefficients are copied into internal holding registers, freeing the caller from having to keep the inputs stable, and the machine advances to a guard step that inspects the leading coefficient. If **a** is zero or is a special encoding (Inf or NaN), the machine diverts to a terminal state that writes a quiet NaN to all six outputs and then signals completion. Otherwise it proceeds into the arithmetic pipeline.

2.3.2 Normalization and Coefficient Build-up

The first arithmetic phase issues the three divisions b/a , c/a , d/a one after another, each held until the divider's pipeline completes, through the shared divider, storing b_a , c_a , d_a . A run of multiplications then forms the products the depressed coefficients need: b_a^2 , b_a^3 , the cross term $b_a c_a$, the shift constant $b_a/3$, and the scaled terms $b_a^2/3$, $2b_a^3/27$, and $b_a c_a/3$. With these in hand, two subtractions and one addition assemble $p = c_a - b_a^2/3$ and $q = (2b_a^3/27 - b_a c_a/3) + d_a$. Each value lands in its own register so that later branches can read it without recomputation.

2.3.3 Discriminant and Dispatch

The discriminant phase squares and cubes p and q as required, scales the terms by $1/27$ and $1/4$, and adds them to form $D = q^2/4 + p^3/27$. Once D is registered, a purely combinational decode of its sign and zero-ness steers the FSM into the Cardano, trigonometric, or repeated-root branch. Only one branch executes per solve; the unused branches are simply never entered.

2.3.4 Branch Execution

Each branch is a sequence of operations matching its formula. The Cardano branch takes the square root of D , forms the two cube root arguments $-q/2 \pm \sqrt{D}$, cube roots them into u and v , and then assembles the real root $u + v$ together with the conjugate pair's real part $-(u + v)/2$ and imaginary magnitude $\frac{\sqrt{3}}{2}(u - v)$. The trigonometric branch builds m and the arccos argument, takes the arccosine, divides the angle by three, and evaluates the cosine three times at the angle and its two $2\pi/3$ offsets, scaling each cosine by m to produce the three real roots. The repeated root branch cube roots $-q/2$ to get r , doubles it for the simple root, and negates it for the double root.

2.3.5 Unshift and Completion

The final phase is shared by every branch. On three successive cycles the stored shift constant $b_a/3$ is subtracted from the real part of each root register, producing the roots of the original equation, while the imaginary parts are forwarded untouched. The machine then enters its completion state, where it asserts **done** for a single cycle with all six output components valid, and returns to idle ready for the next **start** pulse.

3 Internal Implementation

3.1 Overall

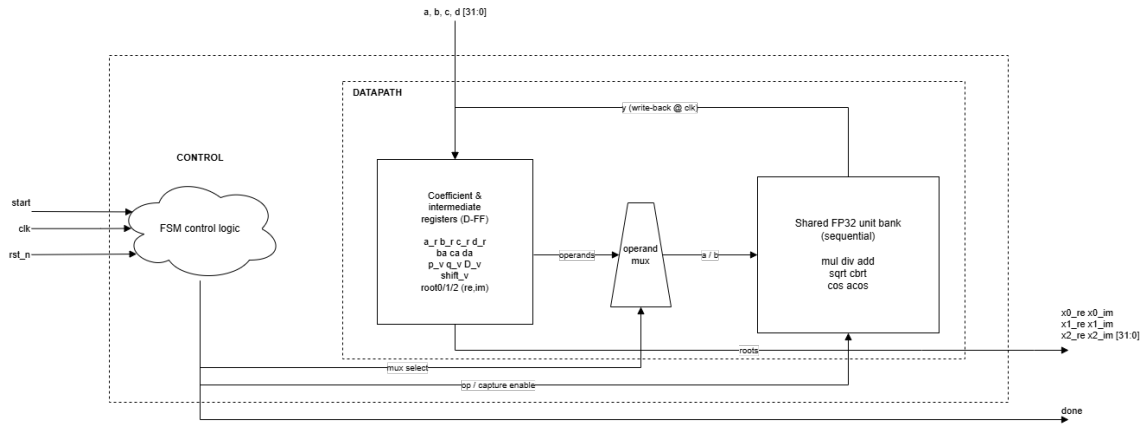


Figure 2: Block Diagram of Cubic Solver

Block / Signal	Description
start	One-cycle strobe that latches the coefficients and begins a solve.
clk	System clock; drives all sequential logic on its rising edge.
rst_n	Active-low reset; forces the FSM to idle and clears all registers.
a, b, c, d [31:0]	The four FP32 cubic coefficients, latched into the register file on launch.
FSM control logic	State register and next-state logic spanning 58 states. Sequences the solve from idle through normalization, depression, discriminant, the selected branch, and the unshift tail. Drives the mux select and the op/capture enable each cycle and asserts done .
Coeff. & intermediate registers	Holds the coefficients, depressed terms (p , q), discriminant, shift constant, and roots.
Operand mux	Combinational multiplexer that selects which registered values are driven onto the shared FP units in the current cycle. Its selection is a pure function of the FSM state.
Shared FP32 unit bank	A single pipelined sequential implementation of each elementary unit: mul, div, add (add/subtract), sqrt, cbrt, cos, and acos, with latencies of 4, 49, 4, 28, 28, 40, and 68 cycles respectively. Reused across states.
y (write-back)	The pipelined output of the active FP unit, routed back to the register file and captured once that unit's latency has elapsed.
done	One-cycle completion flag, asserted by the FSM when all six root components are valid.
x0_re... x2_im	Six FP32 components; three real/imaginary root pairs, stable when done pulses.

Table 3: Block-level description of the `cubic_solver` design

3.2 State Machine

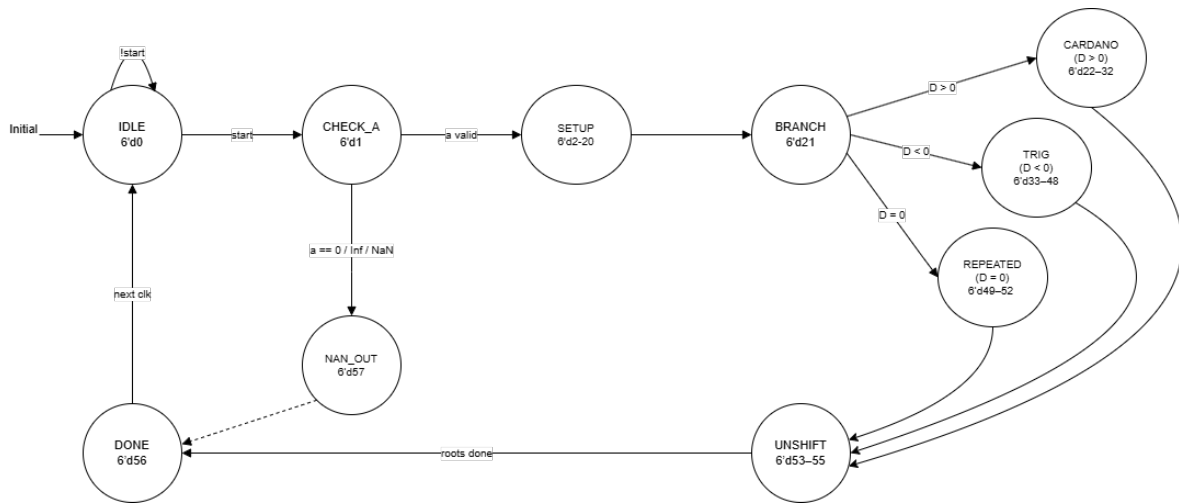


Figure 3: FSM diagram of the `cubic_solver`

The FSM is a single multi-cycle sequence with only two real decision points: the `CHECK_A` guard and the `BRANCH` dispatch on the sign of D . The remaining states are straight-line arithmetic chains, summarised below.

Variable Name	Width	Description
<code>state</code>	6	Current FSM state register, encoded 0–57.
<code>wait_cnt</code>	7	Per-operation latency counter. Counts the cycles the active shared FP unit has been computing in the current state, and triggers result capture when it reaches that unit's pipeline depth.
<code>start</code>	1	Launch strobe. Sampled in <code>IDLE</code> ; HIGH for one cycle latches coefficients and advances to <code>CHECK_A</code> .
<code>a_r</code>	32	Latched leading coefficient. Inspected in <code>CHECK_A</code> ; zero, Inf, or NaN diverts to <code>NAN_OUT</code> .
<code>D_v</code>	32	FP32 discriminant. Sign and zero decoded in <code>BRANCH</code> to select the Cardano, trigonometric, or repeated-root path.

Table 4: Variable descriptions of the `cubic_solver` state machine

State Name	Value	Description
IDLE	6'd0	Default resting state. Holds until start is asserted, then latches a-d and advances.
CHECK_A	6'd1	Validates the leading coefficient. Diverts to NAN_OUT if a zero or subnormal, Inf, or NaN ; otherwise enters SETUP .
SETUP	6'd2-20	19-state linear chain. Normalises by a , computes depressed coefficients p and q , and forms discriminant D .
BRANCH	6'd21	Decodes sign of D and dispatches to CARDANO , TRIG , or REPEATED .
CARDANO	6'd22-32	$D > 0$: computes $\sqrt{D}$, two cube roots u and v , assembles one real root and a complex-conjugate pair.
TRIG	6'd33-48	$D < 0$: builds amplitude m , computes $\theta = \arccos(\cdot)$, evaluates three cosines to yield three real roots.
REPEATED	6'd49-52	$D = 0$: cube-roots $-q/2$ to get r ; emits simple root $2r$ and double root $-r$.
UNSHIFT	6'd53-55	Subtracts $b_a/3$ from each root's real part over three cycles to recover roots of the original equation.
DONE	6'd56	Asserts done for one cycle with all six outputs valid, then returns to IDLE .
NAN_OUT	6'd57	Writes quiet NaN to all six outputs, then passes through DONE .

Table 5: State descriptions of the `cubic_solver` FSM

4 Floating-Point Library

The seven elementary units named in Section 2.1 and listed in Table 6 are each a self-contained, fixed-latency pipeline. Every unit follows the same interface convention: a clock `clk`, a pipeline-enable `en` (held HIGH by the solver so the pipelines never stall), one or two 32-bit operands, and a 32-bit result `y` that becomes valid a fixed number of cycles after the operands are presented.

The adder and multiplier are the foundational blocks; the divider, cosine, and arccosine are built on top of them. The arccosine additionally instantiates the square-root pipeline, so the more complex functions inherit the rounding behaviour of the primitives they call. All five algebraic cores treat an operand whose exponent field is zero as zero (i.e. subnormals are flushed) — the same convention that lets the solver reject a subnormal leading coefficient.

Table 6 summarises the algorithm, latency, and accuracy of each unit; the subsections that follow describe them in turn.

Unit	Method	Latency (cyc)	Accuracy
fp32_add	Align–add–normalise, round-to-nearest-even	4	correctly rounded
fp32_mul	24×24 product, round-to-nearest-even	4	correctly rounded
fp32_div	Newton–Raphson reciprocal (3 iter.) + multiply	49	≈ 1 ulp
fp32_sqrt	Restoring digit recurrence	28	correctly rounded
fp32_cbrt	Restoring digit recurrence (base 3)	28	correctly rounded
fp32_cos	Range reduction + 8th-order Taylor (Horner)	40	$\approx 3 \times 10^{-5}$
fp32_acos	$\sqrt{2u}$ degree-4 polynomial	68	$\approx 0.1\%$

Table 6: Summary of the FP32 primitive units.

4.1 Adder / Subtractor (fp32_add)

The adder is a classic four-stage IEEE-754 add/subtract pipeline. The common path is correctly rounded with an overall latency of four cycles.

- **Stage 1 (Unpack & Compare):** Conditionally negates `b` when `sub` is asserted. It unpacks both operands, flags special cases (NaN, Inf, zero), attaches the hidden mantissa bit, and performs a magnitude comparison to establish the larger exponent as the reference. The alignment shift is recorded as the difference of the exponents.
- **Stage 2 (Align & Compute):** Right-shifts the smaller mantissa by the alignment amount, accumulating a sticky bit from the bits shifted out. It then adds or subtracts the two magnitudes based on whether the effective signs agree.
- **Stage 3 (Normalise):** Normalises the resulting fraction. A carry-out of the top bit shifts the result right by one and increments the exponent. Otherwise, a leading-zero count shifts the result left and decrements the exponent, guarded against underflow.
- **Stage 4 (Round & Pack):** Evaluates Guard, Round, and Sticky bits to enforce round-to-nearest, ties-to-even. It repacks the result and resolves special cases (e.g., any-NaN $\rightarrow$ quiet NaN, $\infty - \infty \rightarrow$ quiet NaN, $\infty + \infty \rightarrow \infty$, exact cancellation $\rightarrow +0$, overflow $\rightarrow \infty$, underflow $\rightarrow 0$).

4.2 Multiplier (fp32_mul)

The multiplier is also a four-stage pipeline, resulting in a correctly rounded product after four cycles.

- **Stage 1 (Unpack):** Unpacks and flags the incoming operands, forming the two 24-bit significands (including the hidden bit).
- **Stage 2 (Multiply):** Forms the full $24 \times 24 \rightarrow 48$ -bit product and calculates the tentative biased exponent via $e_a + e_b - 127$.

- **Stage 3 (Normalise & Round):** A product ≥ 2.0 triggers a one-bit right shift and an exponent increment. The mantissa is then rounded to nearest, ties to even.
- **Stage 4 (Pack):** Packs the 32-bit output vector and resolves special cases: any NaN or $0 \times \infty$ yields a quiet NaN, ∞ yields ∞ , a zero operand yields a signed zero, and exponent under/overflow saturate to 0 or ∞ .

4.3 Divider (fp32_div)

The divider computes a reciprocal by Newton–Raphson iteration and then multiplies, providing an end-to-end latency of 49 cycles with an accuracy of about one ulp.

- **Initialization:** The divisor is normalised to $b' \in [1, 2)$ by forcing its exponent to the bias. A linear seed starts the recurrence:

$$y_0 = 1.5 - 0.5b'$$

- **Newton–Raphson Iteration:** Three iterations of the following error-correction equation are run, which converges quadratically to $1/b'$ and exceeds the 24 bits required by FP32:

$$y_{n+1} = y_n (2 - b' y_n)$$

- **Execution:** Each iteration is a multiply–subtract–multiply triple assembled from the `fp32_mul` and `fp32_add` primitives. The divisor and dividend are carried alongside the datapath in shift registers to remain aligned.
- **Finalization:** The reciprocal's exponent is reconstructed from the iterated mantissa and the original divisor exponent. A final multiply by the dividend produces the quotient. Special cases (NaN, $0/0$, ∞/∞ , $x/0$, x/∞) are resolved at the output.

4.4 Square Root (fp32_sqrt)

The square root uses a restoring digit-recurrence ("shift-and-subtract") scheme containing zero multipliers. Latency is 28 cycles, and the result is correctly rounded.

- **Exponent Pre-calculation:** The result exponent is obtained directly as $E_{\text{new}} = (E + 127) \gg 1$; when the unbiased exponent is odd, the radicand mantissa is shifted left by one to ensure the remaining exponent is even.
- **Digit Recurrence Loop:** Twenty-six recurrence stages generate one mantissa bit per cycle. Each stage brings the next two radicand bits down into a running remainder and compares it against the trial divisor $T = 4Y + 1$ (formed by appending 01 to the partial root Y). If the remainder is large enough, the subtraction is taken and a 1 is appended; otherwise, a 0.
- **Rounding & Specials:** After 26 stages the 24-bit significand is rounded to nearest, ties to even, using the guard and round bits of Y and a sticky bit derived from the final remainder. NaNs or any negative input return a quiet NaN; ± 0 and $+\infty$ return themselves.

4.5 Cube Root (fp32_cbrt)

The cube root reuses the multiplier-free digit-recurrence structure of the square root, adapted to base-3 stepping with a 28-cycle latency.

- **Exponent Pre-calculation:** The exponent is divided by three using a constant-multiply trick: with $A = E + 2$, the quotient $q = \lfloor A/3 \rfloor$ is formed from $A \times 342 (\approx 1024/3)$. The result exponent becomes $q + 84$, while the remainder in $\{0, 1, 2\}$ selects a left-shift of the input mantissa.
- **Digit Recurrence Loop:** Each of the 26 stages brings down three radicand bits and tests the running remainder against $T = 12Y^2 + 6Y + 1$. To avoid squaring Y at every stage, a side register incrementally tracks $3Y^2$, ensuring the entire recurrence strictly uses shifts and adds.
- **Rounding & Output:** The output is rounded to nearest, ties to even. Because the sign is carried through unchanged, the unit safely returns the *real* cube root of a negative argument — exactly what the Cardano branch needs for its $v = \sqrt[3]{-q/2 - \sqrt{D}}$ term.

4.6 Cosine (fp32_cos)

The cosine unit evaluates a truncated Taylor series after an initial argument reduction, yielding a latency of 40 cycles.

- **Argument Reduction:** Since cosine is an even function, the unit evaluates $|a|$. If $|a| > \pi/2$, it calculates $\arg = \pi - |a|$ and stores a flag to negate the final result, ensuring $\arg$ always lies in $[0, \pi/2]$.
- **Taylor Evaluation:** With $u = \arg^2$, it evaluates an eighth-order Taylor polynomial via Horner's rule:

$$\cos(\arg) \approx 1 + u\left(-\frac{1}{2} + u\left(\frac{1}{24} + u\left(-\frac{1}{720} + \frac{u}{40320}\right)\right)\right)$$

- **Buffering & Output:** Evaluated as a chain of multiply and add primitives, u is buffered in delay registers for each Horner step. The delayed sign inversion is then applied. NaN or ∞ inputs force a quiet NaN. The worst-case absolute error on the reduced interval is $\approx 3 \times 10^{-5}$. (Note: The unit assumes $|a| \leq \pi$ as the cubic solver strictly generates arguments in this range).

4.7 Arccosine (fp32_acos)

The arccosine module operates around a square-root-factored polynomial designed to absorb endpoint singularities. It takes 68 cycles (the deepest of any pipeline) with a worst-case error near $x = 0$ of roughly 0.1%.

- **Polynomial Factoring:** Writing $u = 1 - |x|$, the unit evaluates:

$$\arccos(x) = \sqrt{2u} h(u), \quad h(u) = 1 + \frac{u}{12} + \frac{3u^2}{160} + \frac{5u^3}{896} + \frac{35u^4}{18432}$$

- **Singularity Capture:** The infinite slope of arccosine at $x = \pm 1$ is captured by an internal instance of the `fp32_sqrt` pipeline computing $\sqrt{2u}$. This allows the degree-4 polynomial $h(u)$ to converge safely across the entire domain.
- **Clamping & Adjustments:** Inputs whose magnitude slightly exceeds 1 are clamped to the nearest endpoint to tolerate routine trigonometric round-off. The polynomial result is buffered to match the square-root depth before multiplication. For negative arguments, the unit applies $\arccos(x) = \pi - \arccos(|x|)$ at the very end.

4.8 Exponential (fp32_exp2)

The exponential unit computes 2^x using a Padé[3,3] rational approximation, with a latency of 81 cycles (dominated by an internal division).

- **Algorithm:** The unit separates the input x into an isolated integer part $n = \lfloor x \rfloor$ and a positive fractional remainder $f = x - n$. The fraction is centre-shifted into a localised domain $t = f - 0.5$. The numerator polynomial $P(t)$ and denominator $Q(t)$ are evaluated via chained Horner multiplier/adder stages, and pipelined into the `fp32_div` module to find $P(t)/Q(t)$. Finally, the integer n is directly added to the exponent field of the divided mantissa, executing a zero-overhead scale multiplication by 2^n .
- **Legacy & Status:** This module, alongside `fp32_log2`, originally provided an alternative route to algebraic roots via identities like $\sqrt[3]{a} = 2^{\frac{1}{3} \log_2 a}$. However, Version 1 of this design required 4–6 hours to synthesise. The `cubic_solver` datapath was subsequently optimised by replacing it with the dedicated digit-recurrence units detailed in Sections 4.4 and 4.5. It is retained in the codebase as a mathematically sound but unoptimised library function.

4.9 Logarithm (fp32_log2)

The logarithm unit computes $\log_2(x)$ for $x > 0$ using a Padé[3,3] rational approximation, mirroring the 81-cycle latency of the exponential unit.

- **Algorithm:** The approximation relies on the identity $\log_2(x) = (E - 127) + \log_2(m)$, where $m \in [1, 2)$. The exact integer part $(E - 127)$ is converted to FP32. The fractional part $\log_2(m)$ is evaluated using a Padé approximant around $m = 1.5$ (where $t = m - 1.5$), keeping the maximum absolute error to less than one ulp.
- **Legacy & Status:** As with `fp32_exp2`, the logarithm unit was initially implemented to compute cubic and square roots. While removed from the primary solver datapath to drastically reduce synthesis times, it remains available in the library as a valid computation route.